МІНІСТЕРСТВО ОСВІТИ ТА НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»
ФАКУЛЬТЕТ ІНФОРМАТИКИ ТА ОБЧИСЛЮВАЛЬНОЇ ТЕХНІКИ
КАФЕДРА ОБЧИСЛЮВАЛЬНОЇ ТЕХНІКИ

Лабораторна робота №3

з дисципліни «Знайомство із середовищем розробки та складання модульних проектів програм»

на тему ***«*Розробка інтерфейсу користувача*»***

Виконала:
студентка групи ІМ-51
Господінова М.М.
Перевірив:
Рекечинський Дмитро Олександрович

Київ 2026

Лабораторна робота №3. Розробка інтерфейсу користувача
Мета: отримати вміння та навички використовувати інкапсуляцію, абстракцію типів, успадкування та поліморфізм на основі класів, запрограмувавши графічний інтерфейс користувача.

Завдання:
1. Створити у середовищі розробки проект з ім’ям Lab3.
2. Написати вихідний текст програми згідно варіанту завдання.
3. Скомпілювати вихідний текст і отримати виконуваний файл програми.
4. Перевірити роботу програми. За потреби налагодити програму.
5. Проаналізувати та прокоментувати результати та вихідний текст програми.
6. Оформити звіт.

Варіант 6 + 1 = 7:
Кількість об’єктів - 107
Тип масиву - статичний
“Гумовий” слід - пунктирна лінія чорного кольору
Введення прямокутника - від центру до одного з кутів
Прямокутник - чорний контур, світло-зелене заповнення
Введення еліпса - по двом протилежним кутам охоплюючого прямокутника
Еліпс - чорний контур еліпсу без заповнення
Позначення типу фігури - в заголовку вікна

Вихідний код
main.py
```
import tkinter as tk
from tkinter import messagebox

from shapes import Point, Line, Rectangle, Ellipse
from tooltip import ToolTip



class GraphicEditor:
   def __init__(self, root):
       self.root = root
```

```
        self.current_shape = "Point"
        self.root.title(f"Lab3 — Graphic Object Editor —
{self.current_shape}")
        self.root.geometry("900x600")

        self.shapes = []
        self.max_shapes = 107
        self.shapes = [None] * self.max_shapes
        self.shape_count = 0

        self.drawing_shape = self.current_shape

        self.drawing_allowed = True

        self.start_x = 0
        self.start_y = 0

        self.create_menu()
        self.create_toolbar()
        self.create_canvas()
        self.bind_mouse_events()

    def create_menu(self):
        menu_bar = tk.Menu(self.root)

        file_menu = tk.Menu(menu_bar, tearoff=0)
        file_menu.add_command(label="Clear",
command=self.clear_canvas)
        file_menu.add_separator()
        file_menu.add_command(label="Exit",
command=self.root.destroy)
        menu_bar.add_cascade(label="File", menu=file_menu)

        self.objects_menu = tk.Menu(menu_bar, tearoff=0)

        shape_options = [
```

```
            ("Точка", "Point"),
            ("Лінія", "Line"),
            ("Прямокутник", "Rectangle"),
            ("Еліпс", "Ellipse"),
        ]

        self.shape_var = tk.StringVar(value=self.current_shape)

        for label, shape_name in shape_options:
            self.objects_menu.add_radiobutton(
                label=label,
                variable=self.shape_var,
                value=shape_name,
                command=self.select_shape,
            )

        menu_bar.add_cascade(label="Objects",
menu=self.objects_menu)

        help_menu = tk.Menu(menu_bar, tearoff=0)
        help_menu.add_command(label="About",
command=self.show_about)
        menu_bar.add_cascade(label="Help", menu=help_menu)

        self.root.config(menu=menu_bar)

    def create_toolbar(self):
        toolbar = tk.Frame(self.root, bd=1, relief=tk.RAISED)
        toolbar.pack(side=tk.TOP, fill=tk.X)

        buttons = [
            ("Точка", "Point", "Малювати точку"),
            ("Лінія", "Line", "Малювати лінію"),
            ("Прямокутник", "Rectangle", "Малювати прямокутник"),
            ("Еліпс", "Ellipse", "Малювати еліпс"),
        ]
```

```
        for label, shape_name, tooltip_text in buttons:
            button = tk.Button(
                toolbar,
                text=label,
                command=lambda name=shape_name:
self.select_shape_by_name(name),
            )
            button.pack(side=tk.LEFT, padx=3, pady=3)
            ToolTip(button, tooltip_text)

    def select_shape_by_name(self, shape_name):
        self.current_shape = shape_name
        self.shape_var.set(shape_name)
        self.root.title(f"Lab3 — Graphic Object Editor —
{shape_name}")

    def create_canvas(self):
        self.canvas = tk.Canvas(self.root, bg="white",
cursor="cross")
        self.canvas.pack(fill=tk.BOTH, expand=True)

    def select_shape(self):
        self.current_shape = self.shape_var.get()
        self.root.title(f"Lab3 — Graphic Object Editor —
{self.current_shape}")

    def bind_mouse_events(self):
        self.canvas.bind("<Button-1>", self.on_mouse_down)
        self.canvas.bind("<B1-Motion>", self.on_mouse_move)
        self.canvas.bind("<ButtonRelease-1>", self.on_mouse_up)

    def on_mouse_down(self, event):
        self.start_x = event.x
        self.start_y = event.y
        self.drawing_shape = self.current_shape
```

```
        self.drawing_allowed = self.shape_count < self.max_shapes

        if not self.drawing_allowed:
            messagebox.showwarning(
                "Warning",
                f"The limit of {self.max_shapes} objects has been
reached.",
            )
            return

        if self.drawing_shape == "Point":
            shape = Point(event.x, event.y)
            self.shapes[self.shape_count] = shape
            self.shape_count += 1
            shape.draw(self.canvas)

    def on_mouse_move(self, event):
        if not self.drawing_allowed:
            return

        if self.drawing_shape == "Point":
            return

        self.canvas.delete("preview")
        self.draw_preview(event.x, event.y)

    def on_mouse_up(self, event):
        self.canvas.delete("preview")

        if not self.drawing_allowed:
          return

        if self.drawing_shape == "Point":
            return
```

```
        x1 = self.start_x
        y1 = self.start_y
        x2 = event.x
        y2 = event.y

        if self.drawing_shape == "Line":
            shape = Line(x1, y1, x2, y2)

        elif self.drawing_shape == "Rectangle":
            shape = Rectangle(x1, y1, x2, y2)

        elif self.drawing_shape == "Ellipse":
            shape = Ellipse(x1, y1, x2, y2)

        else:
            return

        self.shapes[self.shape_count] = shape
        self.shape_count += 1
        shape.draw(self.canvas)

    def draw_preview(self, x, y):
        x1 = self.start_x
        y1 = self.start_y

        if self.drawing_shape == "Line":
            self.canvas.create_line(
                x1,
                y1,
                x,
                y,
                fill="black",
                width=2,
                dash=(4, 4),
                tags="preview",
            )
```

```
        elif self.drawing_shape == "Rectangle":
            left = 2 * x1 - x
            top = 2 * y1 - y

            self.canvas.create_rectangle(
                min(left, x),
                min(top, y),
                max(left, x),
                max(top, y),
                outline="black",
                fill="",
                width=2,
                dash=(4, 4),
                tags="preview",
            )

        elif self.drawing_shape == "Ellipse":
            left = min(x1, x)
            top = min(y1, y)
            right = max(x1, x)
            bottom = max(y1, y)

            self.canvas.create_oval(
                left,
                top,
                right,
                bottom,
                outline="black",
                fill="",
                width=2,
                dash=(4, 4),
                tags="preview",
            )

    def clear_canvas(self):
```

```
        self.shapes = [None] * self.max_shapes
        self.shape_count = 0
        self.canvas.delete("all")

    def show_about(self):
        messagebox.showinfo(
            "About",
            "Lab3 — Graphic Object Editor.\n" "Developed using
Python and Tkinter.",
        )


if __name__ == "__main__":
    root = tk.Tk()
    app = GraphicEditor(root)
    root.mainloop()
```

shapes.py

```
from abc import ABC, abstractmethod


class Shape(ABC):
    @abstractmethod
    def draw(self, canvas):
        pass


class Point(Shape):
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def draw(self, canvas):
        radius = 3
        canvas.create_oval(
            self.x - radius,
```

```
            self.y - radius,
            self.x + radius,
            self.y + radius,
            fill="black",
            outline="black",
        )


class Line(Shape):
    def __init__(self, x1, y1, x2, y2):
        self.x1 = x1
        self.y1 = y1
        self.x2 = x2
        self.y2 = y2

    def draw(self, canvas):
        canvas.create_line(self.x1, self.y1, self.x2, self.y2,
fill="black", width=2)


class Rectangle(Shape):
    def __init__(self, cx, cy, x, y):
        self.cx = cx
        self.cy = cy
        self.x = x
        self.y = y

    def get_bounds(self):
        left = 2 * self.cx - self.x
        top = 2 * self.cy - self.y

        return (
            min(left, self.x),
            min(top, self.y),
            max(left, self.x),
            max(top, self.y),
```

```
        )

    def draw(self, canvas):
        left, top, right, bottom = self.get_bounds()

        canvas.create_rectangle(
            left,
            top,
            right,
            bottom,
            outline="black",
            fill="lightgreen",
            width=2,
        )


class Ellipse(Shape):
    def __init__(self, x1, y1, x2, y2):
        self.x1 = x1
        self.y1 = y1
        self.x2 = x2
        self.y2 = y2

    def draw(self, canvas):
        x1 = min(self.x1, self.x2)
        y1 = min(self.y1, self.y2)
        x2 = max(self.x1, self.x2)
        y2 = max(self.y1, self.y2)

        canvas.create_oval(
            x1,
            y1,
            x2,
            y2,
            outline="black",
            width=2,
```

```
        )
```

tooltip.py

```
import tkinter as tk



class ToolTip:
    def __init__(self, widget, text):
        self.widget = widget
        self.text = text
        self.tooltip = None

        self.widget.bind("<Enter>", self.show)
        self.widget.bind("<Leave>", self.hide)

    def show(self, event=None):
        if self.tooltip is not None:
            return

        x = self.widget.winfo_rootx() + 20
        y = self.widget.winfo_rooty() + self.widget.winfo_height()
+ 5

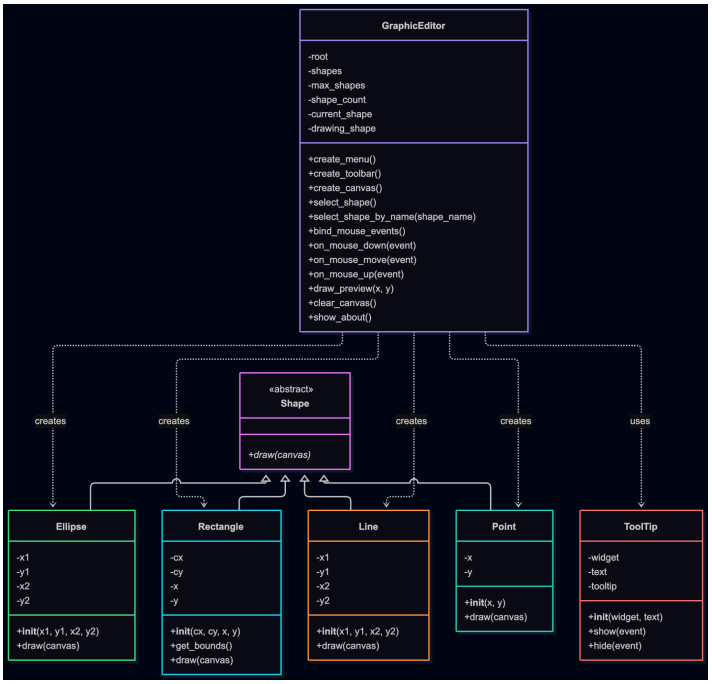
        self.tooltip = tk.Toplevel(self.widget)
        self.tooltip.wm_overrideredirect(True)
        self.tooltip.wm_geometry(f"+{x}+{y}")

        label = tk.Label(
            self.tooltip,
            text=self.text,
            background="#FFF8DC",
            foreground="#222222",
            font=("Arial", 10),
            relief="solid",
            borderwidth=1,
            padx=8,
```

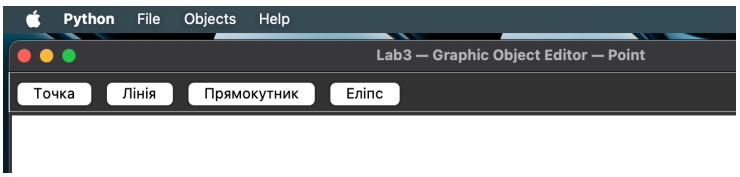
```
            pady=4,
        )
        label.pack()

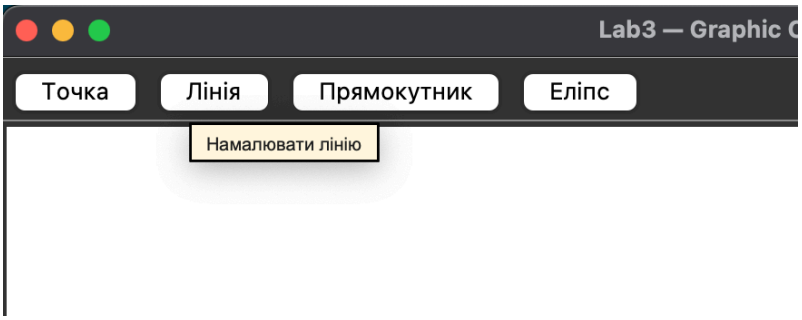
    def hide(self, event=None):
        if self.tooltip is not None:
            self.tooltip.destroy()
            self.tooltip = None
```

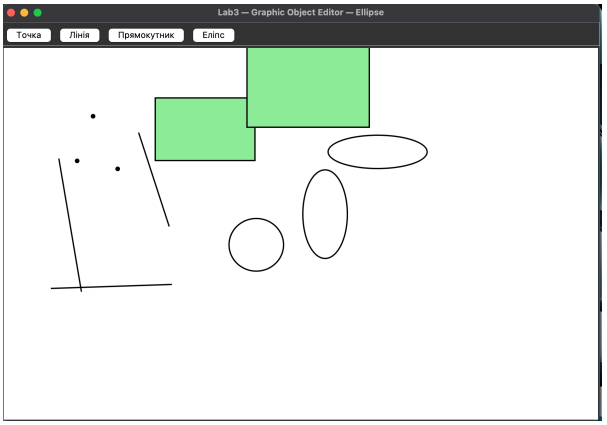
Схема успадкування класів:



Тестування (скріншоти)







Висновок:

Під час виконання лабораторної роботи було розроблено графічний редактор геометричних об’єктів із використанням мови програмування Python та бібліотеки Tkinter. Програма дозволяє малювати на полотні точки, лінії, прямокутники та еліпси. Для реалізації об’єктів було створено абстрактний базовий клас Shape та похідні класи Point, Line, Rectangle і Ellipse.

У програмі реалізовано меню, панель інструментів із підказками, відображення вибраного типу фігури в заголовку вікна та можливість очищення полотна. Під час малювання відображається попередній контур фігури пунктирною лінією. Також передбачено обмеження кількості графічних об’єктів до 107.

У результаті виконання лабораторної роботи було закріплено практичні навички використання принципів об’єктно-орієнтованого програмування, зокрема успадкування, абстракції та поліморфізму. Також було набуто досвіду створення графічного інтерфейсу, обробки подій миші та організації Python-програми за допомогою класів і модулів.